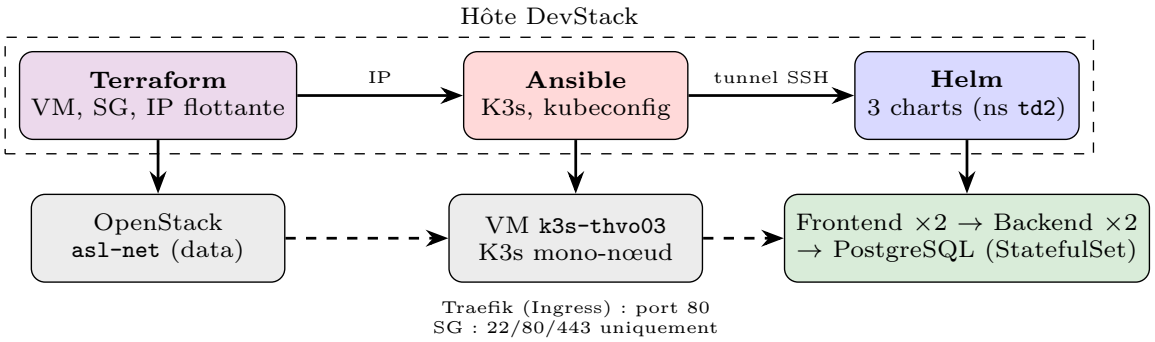
1. Arborescence et architecture

```
stack@openstack-devstack:~/td2$    cd ~/td2 && tree -L 2 -I '.terraform|terraform.tfstate*|terraform.tfvars|kubeconfig|inventory.ini|*.bak'
.
├── README.md
├── ansible
│   ├── ansible.cfg
│   ├── playbook.yml
│   ├── roles
│   └── tunnel.sh
├── helm
│   ├── backend
│   ├── frontend
│   └── postgres
└── terraform
    ├── cloud-init.yaml
    ├── compute.tf
    ├── data.tf
    ├── example.tfvars
    ├── outputs.tf
    ├── providers.tf
    ├── security.tf
    └── variables.tf
```

Arborescence du projet



Traefik (Ingress) : port 80
SG : 22/80/443 uniquement

2. Sorties de validation

```
stack@openstack-devstack:~/td2/terraform$ terraform output
ansible_inventory = <<EOT
[k3s]
k3s-thvo03 ansible_host=172.24.4.222

[k3s:vars]
ansible_user=ubuntu
ansible_password=[redacted]
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
ansible_python_interpreter=/usr/bin/python3

EOT
floating_ip = "172.24.4.222"
private_ip = "10.10.10.140"
ssh_command = "ssh ubuntu@172.24.4.222"
vm_name = "k3s-thvo03"
```

`terraform output`

```
PLAY RECAP *********************************************************************
k3s-thvo03                 : ok=7    changed=4    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
```

PLAY RECAP : premier passage

```
PLAY RECAP *********************************************************************
k3s-thvo03                 : ok=7    changed=0    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
```

PLAY RECAP : second passage idempotent (`changed=0`)

```
stack@openstack-devstack:~/td2/terraform$ helm list -n td2
NAME            NAMESPACE       REVISION        UPDATED
                STATUS          CHART           APP VERSION
backend         td2             4               2026-10-16 14:43:28.161125463
+0000 UTC deployed        backend-0.1.0   1.16.0
frontend        td2             3               2026-10-16 14:32:44.994861943
+0000 UTC deployed        frontend-0.1.0  1.16.0
postgres        td2             2               2026-10-14 13:45:51.954621555
+0000 UTC deployed        postgres-0.1.0  1.16.0
```

`helm list -n td2`

```
stack@openstack-devstack:~/td2/terraform$ kubectl -n td2 get pods,svc,statefulset,
ingress,pvc
NAME                           READY   STATUS    RESTARTS   AGE
pod/backend-665d65c888-7lhzg   1/1     Running   0          8m30s
pod/backend-665d65c888-cs964   1/1     Running   0          8m30s
pod/frontend-f6b9c579b-qd9dj   1/1     Running   0          76s
pod/frontend-f6b9c579b-t6dlw   1/1     Running   0          76s
pod/postgres-0                 1/1     Running   0          11m

NAME               TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
service/backend    ClusterIP   10.43.128.178   <none>        5000/TCP   8m30s
service/frontend   ClusterIP   10.43.70.74     <none>        80/TCP     76s
service/postgres   ClusterIP   None            <none>        5432/TCP   11m

NAME                        READY   AGE
statefulset.apps/postgres   1/1     11m

NAME                                CLASS     HOSTS   ADDRESS        PORTS   AGE
ingress.networking.k8s.io/frontend   traefik   *       10.10.10.140   80      76s

NAME                                        STATUS   VOLUME
          CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/data-postgres-0   Bound    pvc-9c52fef6-23ef-438c-8702-e5027
24ec31a   1Gi        RWO            local-path     <unset>                 11m
```

`kubectl -n td2 get pods,svc,statefulset,ingress,pvc`

```
stack@openstack-devstack:~/td2/terraform$ for i in $(seq 1 6); do curl -sI
"http://$FIP" | grep -i '^X-Pod-Name'; done
X-Pod-Name: frontend-75bd7876ff-bbnbt
X-Pod-Name: frontend-75bd7876ff-mz7gn
X-Pod-Name: frontend-75bd7876ff-bbnbt
X-Pod-Name: frontend-75bd7876ff-mz7gn
X-Pod-Name: frontend-75bd7876ff-bbnbt
X-Pod-Name: frontend-75bd7876ff-mz7gn
```

Header `X-Pod-Name` alternant

```
stack@openstack-devstack:~/td2/terraform$ curl -s "http://$FIP/api/items" |
 python3 -m json.tool
{
    "pod": "backend-5bcbd74c74-lv9xd",
    "items": [
        {
            "id": 1,
            "name": "item-alpha"
        },
        {
            "id": 2,
            "name": "item-beta"
        },
        {
            "id": 3,
            "name": "item-gamma"
        }
    ]
}
```

`curl -s http://<FIP>/api/items`

```
user@debian:~$ ssh -L 8080:172.24.4.222:80 ubuntu@10.0.0.100
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-142-generic x86_64)
```



Interface NovaShop (tunnel SSH, `localhost:8080`)

3. Questions de réflexion

Q1 - Data sources pour `asl-net`. Le réseau, la clé et l'image viennent du TD1 et sont partagés : Terraform doit les *lire* sans les posséder. Avec une `resource`, il tenterait de recréer un réseau existant (conflit) et surtout un `terraform destroy` supprimerait le réseau commun. Inversement, déclarer le security group en `data` échouerait car il n'existe pas encore.

Q2 - Frontière Terraform / Ansible. Terraform provisionne les objets de l'API OpenStack (VM, SG, IP flottante) et suit leur état. Ansible configure l'intérieur de la VM (paquets, fichiers, service K3s). Terraform ne peut pas installer K3s ni écrire `/etc/rancher/k3s/config.yaml`, et Ansible ne gère pas proprement le cycle de vie (plan/destroy) d'une IP flottante.

Q3 - StatefulSet vs Deployment. (1) Identité stable : `postgres-0` garde son nom et son DNS, alors que les pods d'un Deployment ont des noms aléatoires. (2) `volumeClaimTemplates` crée un PVC propre à chaque réplica (`data-postgres-0`) qui est rattaché à nouveau après un redémarrage ; en plus, démarrage et arrêt sont ordonnés.

Q4 - Tunnel SSH. Le port 6443 donne un accès d'administration complet au cluster ; l'ouvrir à `0.0.0.0/0` agrandirait fortement la surface d'attaque. Le tunnel réutilise le port 22 déjà autorisé et authentifié, chiffre le trafic et n'ajoute aucune règle au SG (moindre privilège).

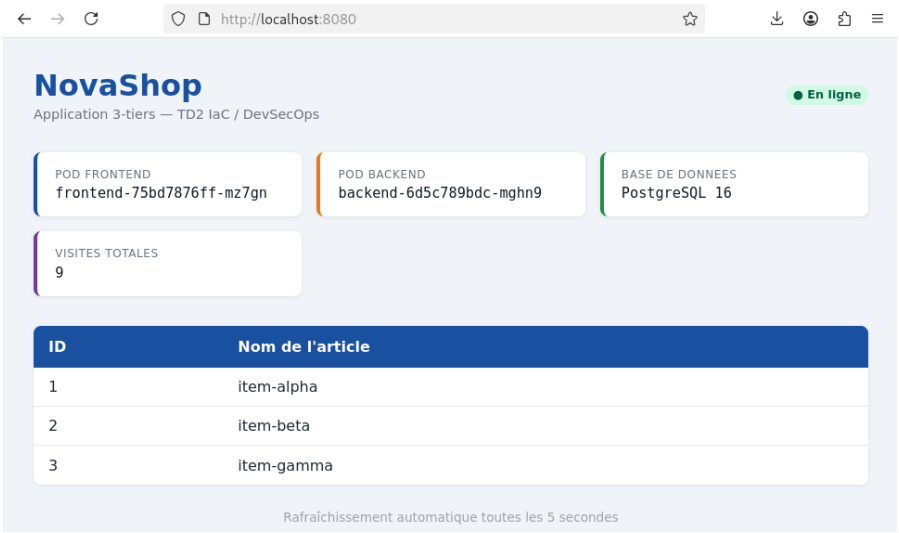
Q5 - Après `terraform destroy`. Terraform détruit 7 ressources (SG, 3 règles, VM, IP flottante, association). Subsistent : projet et utilisateur `asl`, réseau `asl-net` et son sous-réseau, routeur `asl-router`, réseau `public`, paire de clés `asl-key`, image et flavor. C'est voulu : ce sont des ressources partagées, lues par des `data`, qu'une destruction applicative ne doit jamais casser.

```
Destroy complete! Resources: 7 destroyed.
stack@openstack-devstack:~/td2/terraform$
```

`terraform destroy` : 7 ressources détruites

4. Exercice bonus - Compteur de visites

L'exercice bonus (étape 15) a été réalisé : table `visits` créée à la main, endpoint `/api/visits` au backend et card « Visites totales » au frontend (capture ci-dessous). Cycle complet : `helm upgrade` (révision 3), `rollout restart`, puis `helm rollback` (révision 4) suivi d'un `rollout restart` : `/api/visits` renvoie bien 404. Un rollback ne redémarre pas les pods quand seul un ConfigMap change ; une annotation `checksum/code` l'automatiserait.



Interface NovaShop avec la card « Visites totales »

```
stack@openstack-devstack:~/td2/helm$ helm history backend -n td2
REVISION        UPDATED                         STATUS          CHART           APP VERSION     DESCRIPTION
1               Wed Oct 14 13:23:23 2026        superseded      backend-0.1.0   1.16.0          Install complete
2               Wed Oct 14 13:45:56 2026        superseded      backend-0.1.0   1.16.0          Upgrade complete
3               Fri Oct 16 14:32:36 2026        deployed        backend-0.1.0   1.16.0          Upgrade complete
```

`helm history backend -n td2` (3 révisions)

```
stack@openstack-devstack:~/td2/helm$ kubectl -n td2 rollout status deployment/backend
Waiting for deployment "backend" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "backend" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "backend" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "backend" rollout to finish: 1 old replicas are pending termination...
deployment "backend" successfully rolled out
```

`kubectl rollout status deployment/backend`

```
stack@openstack-devstack:~/td2/helm$ helm history backend -n td2
REVISION        UPDATED                         STATUS          CHART           APP VERSION     DESCRIPTION
1               Wed Oct 14 13:23:23 2026        superseded      backend-0.1.0   1.16.0          Install complete
2               Wed Oct 14 13:45:56 2026        superseded      backend-0.1.0   1.16.0          Upgrade complete
3               Fri Oct 16 14:32:36 2026        superseded      backend-0.1.0   1.16.0          Upgrade complete
4               Fri Oct 16 14:43:28 2026        deployed        backend-0.1.0   1.16.0          Rollback to 2
```

`helm history backend -n td2` après rollback (4 révisions)

```
stack@openstack-devstack:~/td2/helm$ curl -sI "http://$FIP/api/visits" | head -1
HTTP/1.1 501 Not Implemented
stack@openstack-devstack:~/td2/helm$ curl -s -o /dev/null -w "%{http_code}\n" "http://$FIP/api/visits"
404
```

`/api/visits` renvoie 404 après rollback

Question de synthèse (migration avant le backend). Un Job Helm `pre-upgrade` (`helm.sh/hook`) exécute la migration avant le déploiement du nouveau backend. Un `initContainer` par pod serait risqué avec 2 réplicas (migrations concurrentes) ; il ne sert qu'à attendre la base. Flyway ou Liquibase garantissent l'ordre et l'historique.

Q bonus - Pourquoi `init.sql` ne rejoue pas ? L'image Postgres n'exécute `docker-entrypoint-initdb.d` que si le répertoire de données est vide. Après `helm upgrade`, le PVC contient déjà des données.